

"News, rumors, revelations,
and prophecies from the age
of artificial intelligence."

The Ai Times

Editorial Board:
Alshival.Ai



Samuel Cavazos
Top 100 Data & Analytics Professional

Large Language Regression Models

Last year, a medical institution reached out to me with a very unique problem. They had a program that provided funding to undergraduate students conducting medical research. Students would submit a research proposal, and a review committee composed of some of the world's best medical researchers would evaluate the submissions. The problem was that members of the review committee are paid for their time, and the number of applications made a full manual review financially impossible. How could they efficiently evaluate all the proposals without overspending on reviewer compensation?

Their researchers had tried to use large language models for this task, hoping that the models could help evaluate the research proposals and reduce the burden on human reviewers. However, they found results were inconsistent and unreliable. I spoke with them about how large language models are designed for *sequence-to-sequence* tasks rather than a *regression* task like scoring research proposals, and the institution contracted me to do some research and see how far we could get.

They sent over a bunch of PDF files, so I had to do some preprocessing to extract the data. That was surprisingly difficult because college students. Some students used non-standard encodings, so when I extracted the text, the data was corrupted. Others submitted images of their proposal. That one made me laugh. The students would scan a printed hard-copy of their proposal and submit the scans in a PDF. In that case, text extraction fails because the PDF is essentially an image with no underlying text. I ended up having to use optical character recognition to get the text out of those image-based PDFs.

Once I had the training data, I spent a few weeks thinking about how to proceed. I had all this text. What could I do with it?

After a few hours at a whiteboard, I

sketched out a plan for a class of models I now refer to as *large-language regression models* (LLRMs). It was perfect for this kind of *sequence-to-number* task.

The Four-Tensor Twist

The initial LLRM I designed was based on the transformer architecture, with a twist. Instead of a single input tensor, I had four tensors of different dimensions. The *title tensor* was an embedding of the project title. The *abstract summary tensor* was another embedding, constructed by embedding the entire abstract at once. These are 1-dimensional tensors. In addition, I had two 2-dimensional tensors that had to be blended into the same architecture. Instead of embedding the abstract entirely, each sentence was embedded and the collection of these embeddings formed the *abstract-sentence tensor*, a matrix of up to 50 sentence embeddings, each with 3,072 dimensions. Similarly, embedding each sentence of the proposal produced the *proposal-sentence tensor*, a matrix of up to 512 sentence embeddings, each with 3,072 dimensions.

Blending the Four Tensors

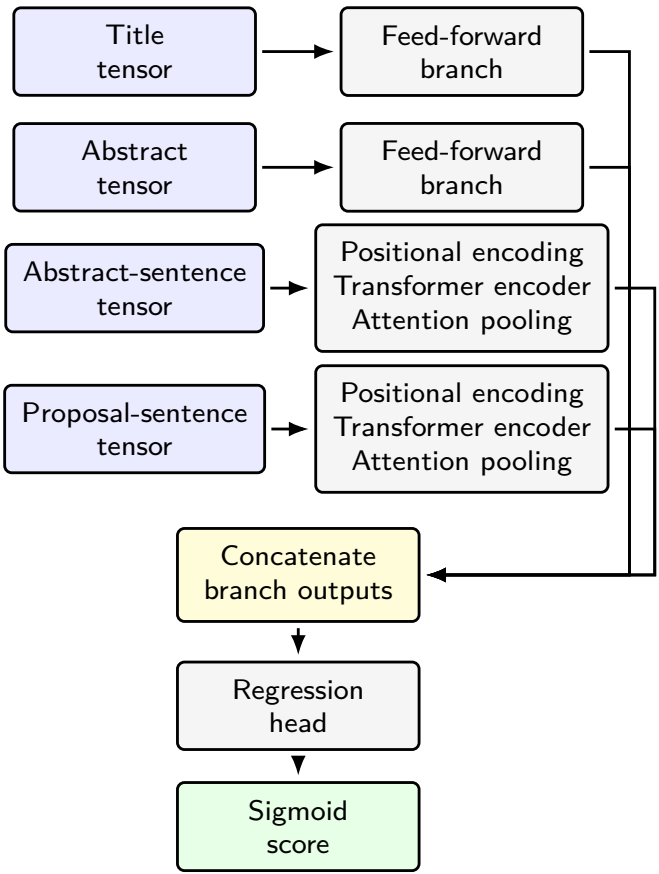
To combine the four tensors, I designed separate branches for each input. The title and abstract tensors went through dense layers to reduce their dimensionality. The abstract-sentence and proposal-sentence tensors went through positional encoding, transformer encoders, and attention pooling to produce fixed-length vectors. Finally, the outputs of all four branches were concatenated and fed into a regression head to produce the final score.

The model took about 6 to 12 hours to train on my RTX 4080. I remember laughing when I saw the results. 86% of the test data was accurately classified when we used a threshold to determine whether an application makes the short-list. I was curious though how it would do on completely new applications.

Well, they tried it last year, and results were so good that the client asked us to retrain the model for this year's set of applications. This past weekend, we

trained on an RTX 5070 and were able to get up to 90% accuracy on the test set. I had one of my data science interns do most of the heavy lifting this time so that they can take over the model next year.

LLRMs do not have the generative capabilities of LLMs; they are designed purely for producing a numerical score from text inputs rather than generating new sequences of text and that is what they excel at. An important model class that gets very little attention compared to LLMs, but has significant practical value in tasks involving evaluation, scoring, and filtering of text. The next time a sequence-to-number project lands on my desk, I will be ready.



Schematic of the full four-branch LLRM architecture.

Announcements

The Ai Times is now on Substack. Scan the code below to get each new issue, along with the sources, side quests, and stray prophecies that did not fit on the page.

THE
AI
TIMES

